

并行处理与体系结构(7) Parallel Processing and Architecture

2025-05-18

1

READING LIST (3)

ZY2457318	戚永飞
ZY2457328	于洁洁
ZY2457329	张超骏
ZY2457415	刘鑫

Topic 3

=====

Prefetching 第12周 (5月18日下午)

3a) T. Mowry et al. "Design and Evaluation of a Compiler Algorithm for Prefetching", Architectural Support for Programming Languages and Operating Systems, 1992.

3b) Y. Solihin et al. "Using a User-Level Memory Thread for Correlation Prefetching", International Symposium on Computer Architecture 2002.

2

2

READING LIST (4)

ZY2457308	王康宁
ZY2457311	王振伟
ZY2457312	王志飞
ZY2457316	文砚雷

Topic 4

=====

Synchronization 第12周 (5月18日下午)

4a) J. Goodman et al. "Efficient Synchronization Primitives for Large Scale Cache-Coherent Multiprocessors", Architectural Support for Programming Languages and Operating Systems, 1989.

4b) J. Mellor-Crummey and M. Scott. "Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors", ACM Transactions on Computer Systems, 1991

3

3

READING LIST (5)

ZY2457213	李家兵
ZY2457216	李尚原
ZY2457229	邱文凯
ZY2457233	任星舟

Topic 5

=====

Multithreading 第13周 (5月25日上午)

5a) D. Tullsen et al. "Simultaneous multithreading: Maximizing On-Chip Parallelism", International Symposium on Computer Architecture 1995.

5b) D. Tullsen et al. "Exploiting Choice: Instruction Fetch and Issue on an Implementable Simultaneous Multithreading Processor", International Symposium on Computer Architecture 1996.

4

4

READING LIST (6)

Topic 6

=====

Multiple Processors on a Chip 第13周 (5月25日上午)

- 6a) K. Olukotun et al. "The Case for a Single-Chip Multiprocessor", Architectural Support for Programming Languages and Operating Systems, 1996
6b) G. Sohi et al. "Multiscalar Processors", International Symposium on Computer Architecture 1995.
6c) V. Krishnan and J. Torrellas. "A Chip Multiprocessor Architecture with Speculative Multithreading", IEEE Transactions on Computers 1999

ZY2457118	程骁
ZY2457124	段泽邦
ZY2457129	郭燕
ZY2457206	沈紫静

▶ 5

5

READING LIST (7)

Topic 7

=====

Speculative Parallelization and Execution 第13周 (5月25日下午)

- 7a) J. Steffan et al. "A Scalable Approach to Thread-Level Speculation" International Symposium on Computer Architecture 2000.
7b) J. Martinez et al. "Speculative Synchronization: Applying Thread-Level Speculation to Explicitly Parallel Applications", Architectural Support for Programming Languages and Operating Systems, 2002.

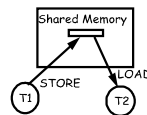
ZF2406102	盖一鸣
ZF2406108	罗晖
ZF2406114	张峻玮
ZY2457103	段欣然
ZY2457104	姜明月

▶ 6

6

回顾：同步

- ▶ 片上多处理器会共享部分内存
- ▶ 通信隐式的通过LOAD和STORE实现
共享内存是一种通信机制
- ▶ 需要“互斥”
 - ▶ 必须使程序语句的执行表现出原子性使用硬件原语同步
- ▶ 同步的方法
 - ▶ 获取方法
 - ▶ 释放方法
 - ▶ 等待算法
 - ▶ 阻塞
 - ▶ 忙等
- ▶ BARRIER 同步
 - ▶ 在所有线程中实现全局同步
- ▶ 锁
 - ▶ ISA 支持: 很多现代机器采用一种RMW的原子操作指令



▶ 7

7

回顾：软件锁

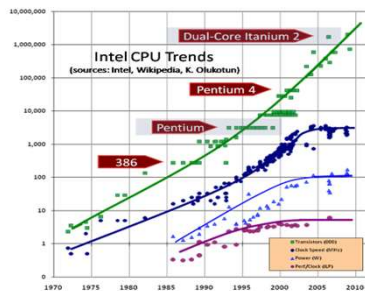
- ▶ 问题: 锁不是原子的—两个线程能同时获得锁
- ▶ 解决: 原子的读/写或交换指令
 - ▶ 原子地从某个位置读取值, 并且向该位置写新值
- ▶ 最简单的一个: TEST_AND_SET (T&S)
- ▶ 其它的 R/M/W 原子操作
 - ▶ SWAP RI, MEM_LOC: 交换 RI 和 MEM_LOC 的内容
 - ▶ FETCH&OP
 - ▶ COMPARE&SWAP
- ▶ 降低T&S的频率
 - ▶ 带退避的T&S
 - ▶ TEST AND TEST&SET 锁
- ▶ 加锁读或关联读 (LL)和有条件写 (SC)
 - ▶ 在流水线中更容易实现
- ▶ 灵活

▶ 8

8

回顾：单处理器的问题

- ▶ 复杂性
- ▶ 新增硬件的边际效用
- ▶ 能耗
- ▶ 缺少指令级并行



▶ 9

9

回顾：两种选择

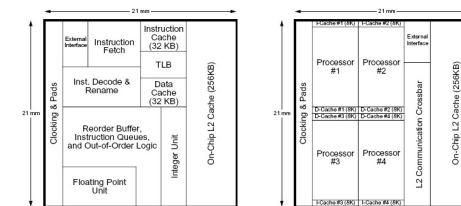


Figure 2. Floorplan for the six-issue dynamic superscalar microprocessor.

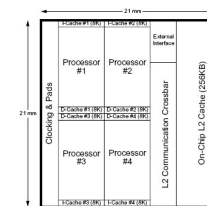


Figure 3. Floorplan for the four-way single-chip multiprocessor.

- ▶ [Olukotun 1996] Multithreading
Multicore (Chip Multiprocessing)

▶ 10

10

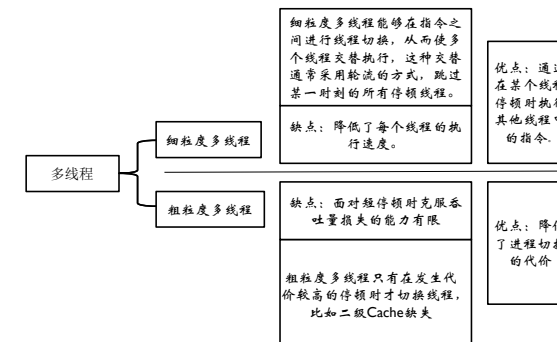
回顾：并行的分类

- ▶ 指令级并行 (ILP)
 - 独立的指令之间
 - 硬件只能通过指令窗口观察到ILP
 - 编译器可以重新组织指令以使得落入指令窗口的指令相对独立
- ▶ 线程级并行 (TLP)
 - 编译器将程序切分成多个控制线程 (每个线程执行一组指令)
 - 不需要一个大的窗口 (每个线程可以关注一个小的窗口)

▶ 11

11

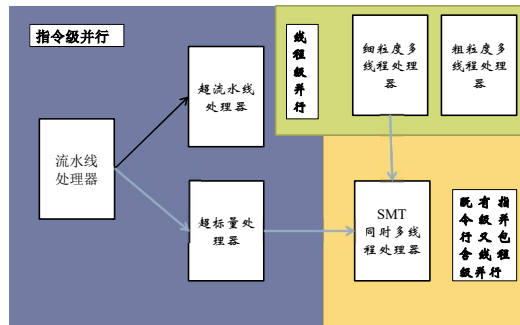
回顾



▶ 12

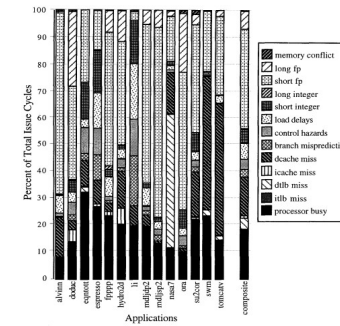
12

回顾



13

回顾：时钟周期都去哪儿了？



一个八发射的超标量处理器。CPU只有平均19%的利用率，即 $IPC < 1.5$

14

回顾：SMT的动机

- ▶ 宽发射的超标量结构仍然可行
 - ▶ 有必要继续提升单一程序的性能
- ▶ 多道程序共享取指令和发射带宽
 - ▶ TLP改善了宽发射超标量的吞吐
 - ▶ 仍然可以利用单一程序中较高的ILP

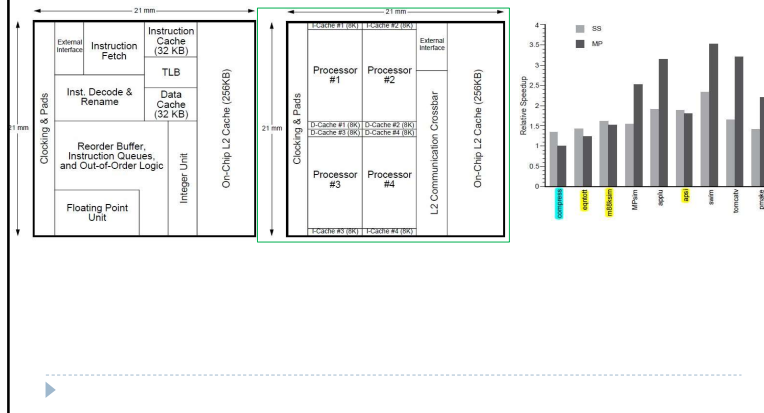
15

回顾：CMP（多核）的动机

- ▶ 宽发射超标量并非解决之道
 - ▶ 很高的时钟频率VS很小的IPC增加
 - ▶ 设计和验证的复杂性
- ▶ 多个简单的处理器
 - ▶ 利用TLP
 - ▶ 适度的ILP加上高频率时钟

16

回顾



17

回顾：TLS（线程级投机）

- ▶ 主要思路：并行的执行不安全的线程
 - ▶ 通常是来自串行程序中的线程
 - ▶ 硬件监控数据冲突
 - ▶ 冲突发生时，清除并重启相关线程
 - ▶ 不发生冲突
 - ▶ 线程执行完毕
 - ▶ 一个线程前序线程都执行完毕，该线程变为非投机或者安全线程
 - ▶ 前序线程全部提交后，该线程提交
- 线程按序提交

18

回顾：非传统的并行

- ▶ 并行性—使用多个上下文获得好于一个上下文所能获得的性能
- ▶ 传统的并行性—使用附加的线程/处理器分配计算：线程分割执行流
- ▶ 非传统的并行性—附加线程用于加速计算，不需要分配原始计算
 - ▶ 最大的优势：几乎对所有代码，不管它的可并行性如何，都能从并行中获得益
 - ▶ 其他的优点：可以灵活地增加或减少线程而不会导致明显的执行中断

19

回顾：Helper 线程

- ▶ **Helper线程预取**
 - ▶ 由程序员/编译器生成
 - ▶ 由硬件生成
- ▶ **Helper线程改善代码质量(重编译)**

20

回顾：并行性

▶ Amdahl定律

- ▶ f: 程序中可并行的部分
- ▶ N: 处理器个数

$$\text{加速比} = \frac{1}{1-f + \frac{f}{N}}$$

- ▶ Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

▶ 最大加速比受限于串行部分: 串行瓶颈

▶ 并行部分通常不完美

- ▶ 同步开销 (比如, 对共享数据的更新)
- ▶ 负载不均衡开销 (并行化不完美)
- ▶ 资源共享开销 (N个处理器之间的竞争)

21

问题: 串行化的代码段

▶ 很多并行程序无法完全并行化

▶ 串行化代码段的产生

- ▶ 连续的部分(Amdahl的“串行部分”)
- ▶ 临界区
- ▶ 栅障
- ▶ 流水化程序中的受限阶段

▶ 串行化的代码段

- ▶ 降低性能
- ▶ 限制扩展性
- ▶ 浪费能源

22

回顾：不同代码段的需求

▶ 我们想要:

▶ 串行代码段 → 一个强有力的“大”核

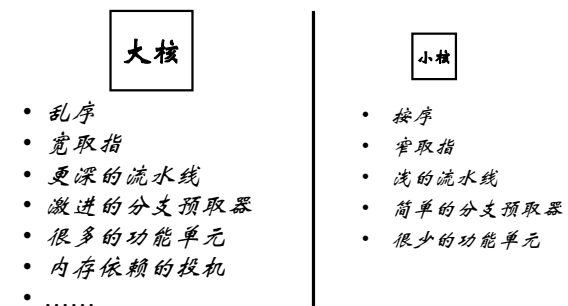
▶ 并行代码段 → 许多弱的“小”核

▶ 这两者互相冲突:

- ▶ 如果你有一个强有力的核, 就不可能同时拥有很多核
- ▶ 一个小孩在能耗和面积消耗方面都远比一个大核更高效

23

回顾：“大” vs. “小” 核

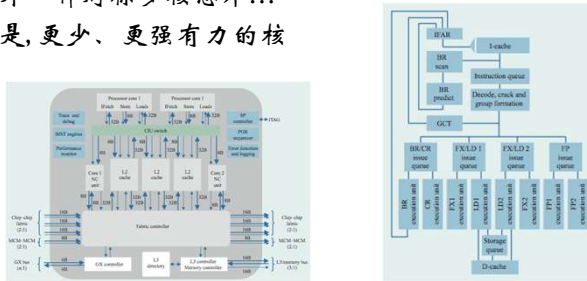


大核的能效低:
比如, 4X的面积(功率)只能获得2X的性能

24

大核: IBM POWER4

- ▶ Tendler et al., “POWER4 system microarchitecture,” IBM J R&D, 2002.
- ▶ 另外一种对称多核芯片...
- ▶ 但是, 更少、更强有力的核



IBM POWER4

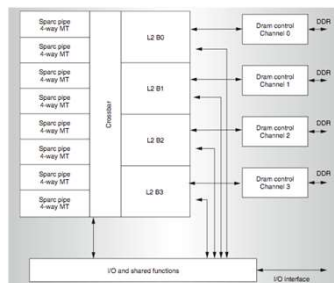
- ▶ 2 个核, 乱序执行
- ▶ 每核 100-entry 的指令窗口
- ▶ 8 宽度取指、发射和执行
- ▶ 大容量, 本地+全局混合分支预测器
- ▶ 1.5MB, 8路 L2 cache
- ▶ 基于流的激进预取

25

26

小核: Sun Niagara (UltraSPARC T1)

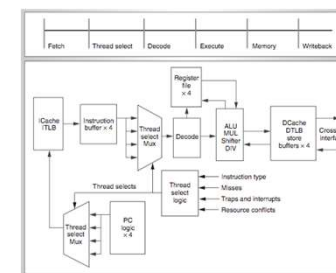
- ▶ Kongetira et al., “Niagara: A 32-Way Multithreaded SPARC Processor,” IEEE Micro 2005.



27

Niagara的核

- ▶ 4路细粒度多线程, 6阶段双发射按序执行
- ▶ 循环线程选择(除非发生cache缺失)
- ▶ 核间共享FP单元



28

回顾：需求

- ▶ 我们要：
- ▶ 串行代码段→一个强有力的“大”核
- ▶ 并行代码段→许多弱的“小”核
- ▶ 这两者互相冲突：
 - ▶ 如果你有一个强有力的核，就不可能同时拥有很多核
 - ▶ 一个小核在能耗和面积消耗方面都远比一个大核更高效
- ▶ 能否两全其美？

29

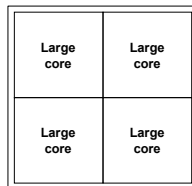
性能 vs. 并行性

假设：

1. 小核用1个面积的预算获得1份性能
2. 大核用4个面积的预算获得2份性能

30

铺砌大核

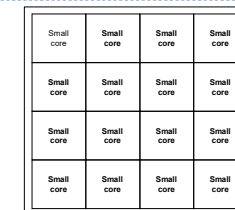


“铺砌大核”

- ▶ 铺砌少量大核
- ▶ IBM Power 5, AMD Barcelona, Intel Core2Quad, Intel Nehalem
- + 单线程、串行代码段时可获得高性能
- 并行程序段时吞吐率低

31

铺砌小核



“铺砌小核”

- ▶ 铺砌很多小核
- ▶ Sun Niagara, Intel Larrabee, Tilera TILE
- + 并行部分吞吐率高
- 串行部分、单线程性能低

32

两全其美?

► 铺砌大核

- + 单线程、串行代码段时可获得高性能
- 并行程序段时吞吐率低

► 铺砌小核

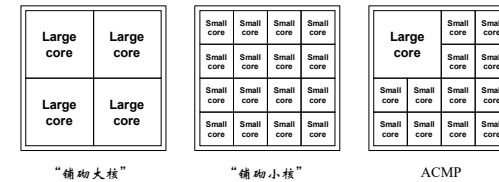
- + 并行部分吞吐率高
- 串行部分、单线程性能低, 相比现有单线程处理器性能还差

► 思路: 在一个芯片上同时集成大核和小核 → 性能不对称



33

非对称片上多处理器(ACMP)



► 提供一个大核和多个小核

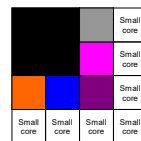
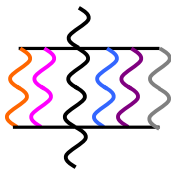
- + 利用大核加速串行部分
- + 在小核和大核上执行并行部分以获得高的吞吐率



34

加速串行瓶颈

单线程 → 大核



ACMP



35

性能 vs. 并行性

假设:

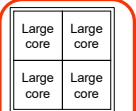
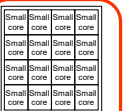
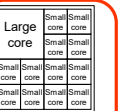
1. 小核用1个面积的预算获得1份性能
2. 大核用4个面积的预算获得2份性能



36

ACMP 性能 vs. 并行性

面积预算 = 16 个小核

			
	“确凿大核”	“确凿小核”	ACMP
大核	4	0	1
小核	0	16	12
串行性能	2	1	2
并行吞吐	$2 \times 4 = 8$	$1 \times 16 = 16$	$1 \times 2 + 1 \times 12 = 14$

37

再审视：并行性

▶ Amdahl 定律

- ▶ f: 程序中可并行的部分
- ▶ N: 处理器个数

$$\text{加速比} = \frac{1}{1 - f + \frac{f}{N}}$$

- ▶ Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” AFIPS 1967.

▶ 最大加速比受限于串行部分: 串行瓶颈

▶ 并行部分通常不完美

- ▶ 同步开销 (比如, 对共享数据的更新)
- ▶ 负载均衡开销 (并行化不完美)
- ▶ 资源共享开销 (N个处理器之间的竞争)

38

加速并行瓶颈

- ▶ 并行部分中的序列化或不均衡的执行同样可以得益于大核

▶ 例子:

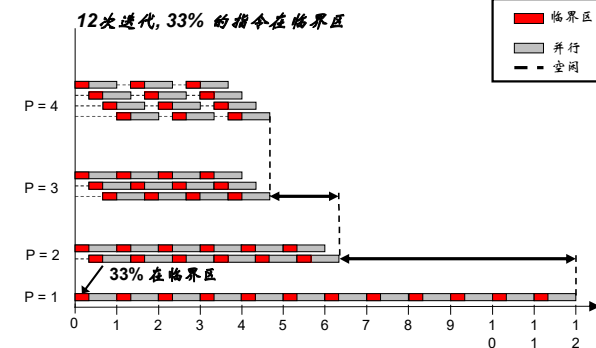
- ▶ 临界区竞争
- ▶ 比别的阶段执行时间更长的并行阶段

▶ 思路: 动态判别会导致序列化执行的代码段并将它们放到 大核上执行

- ▶ 加速临界区
- ▶ 瓶颈识别和调度

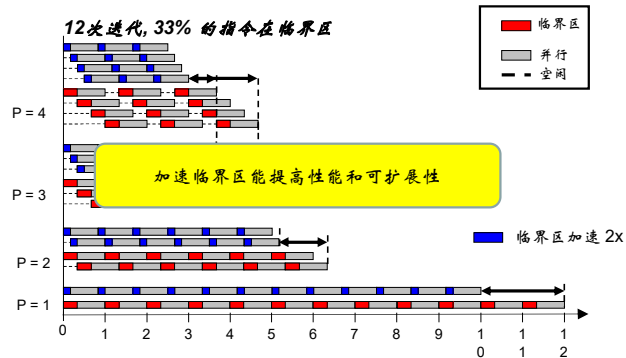
39

临界区竞争



40

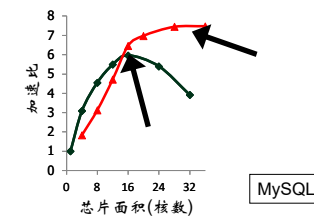
临界区竞争



41

临界区对可扩展性的影响

- ▶ 临界区竞争导致并行程序段中线程的串行执行(序列化)
- ▶ 临界区竞争随着线程数增加而增加, 并且限制可扩展性



42

利用不对称

- ▶ 串行部分的执行时间必须短
- ▶ 对程序员来说缩短这些串行段相当困难
 - ▶ 领域知识不够
 - ▶ 硬件平台的多样性
 - ▶ 受限的资源
- ▶ 目标: 一种不需要程序员参与的缩小串行瓶颈的机制
- ▶ 思路: 在非对称多核平台上通过将串行代码段迁移到强有力的核上来加速串行部分的执行

43

一个例子: 加速临界区

- ▶ 思路: 在非对称多核体系结构中将临界区迁移到大的、强有力的核上
- ▶ 好处:
 - ▶ 减少由于锁的争用带来的串行化
 - ▶ 减少无法并行化的部分对性能的影响
 - ▶ 程序员不需要(过多地)优化并行代码 → 更少的bug, 提高效率

44

回顾：多线程应用中的瓶颈

一种定义：任何会被线程竞争的代码段

▶ Amdahl的串行段

- ▶ 只有一个线程在执行 → 在关键路径上

▶ 临界区

- ▶ 保证互斥 → 如果有竞争很可能就在关键路径上

▶ 栅障

- ▶ 再继续推进之前保证所有线程到达该点 → 最后到达的线程在关键路径上

▶ 流水线阶段

- ▶ 一个循环迭代的阶段可能在不同线程上执行，最慢的阶段会使其它阶段等待 → 在关键路径上

45

45

有什么收获？

- ▶ 硬件并行在很多方面都得到发展

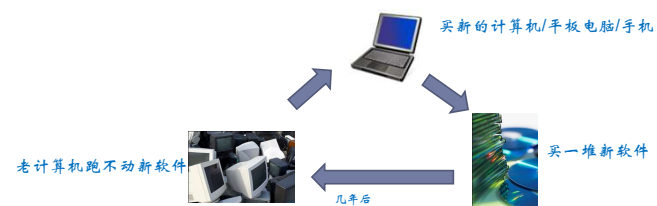
- ▶ 能够获得更大的性能提升？

▶

46

性能提升？－问题何在？

- ▶ 处理器性能提升驱动着硬件**周期性升级**



- ▶ 硬件周期性升级驱动着巨大的经济利益

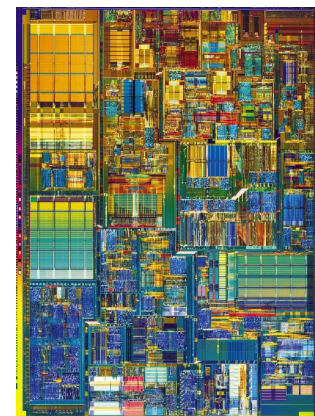
▶

47

是什么限制了处理器性能？

▶ 传统上

- ▶ 晶体管
- ▶ 功率
- ▶ 软件

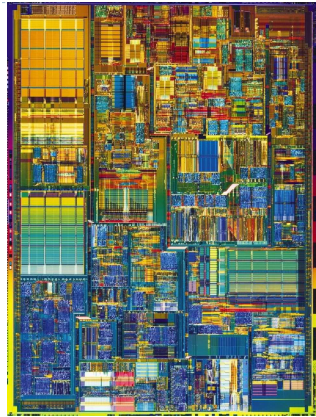


▶

48

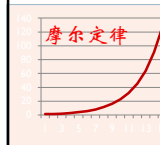
处理器性能瓶颈?

晶体管 →
功率 →
软件 →

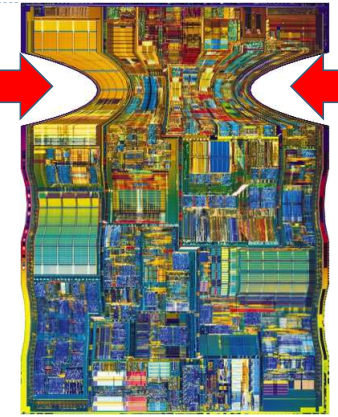


49

处理器性能瓶颈?



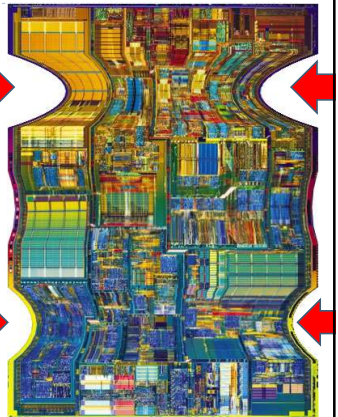
晶体管 →
功率 →
软件 →



50

处理器性能瓶颈?

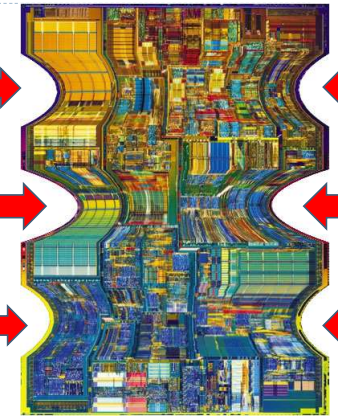
晶体管 →
功率 →
软件 →



51

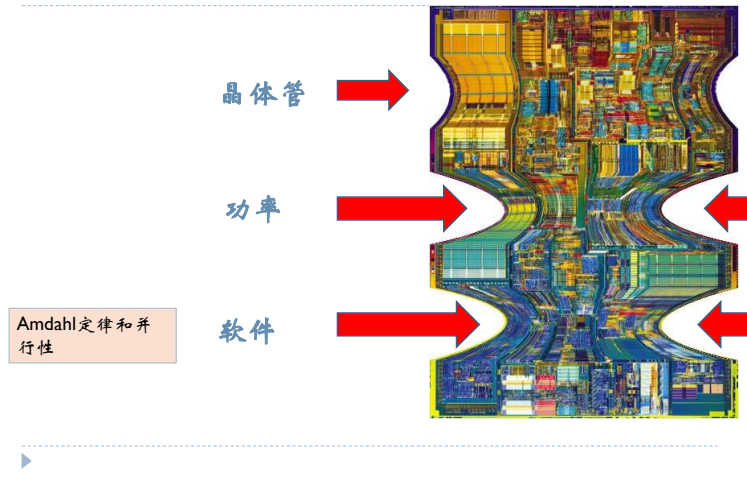
处理器性能瓶颈?

晶体管 →
功率 →
软件 →



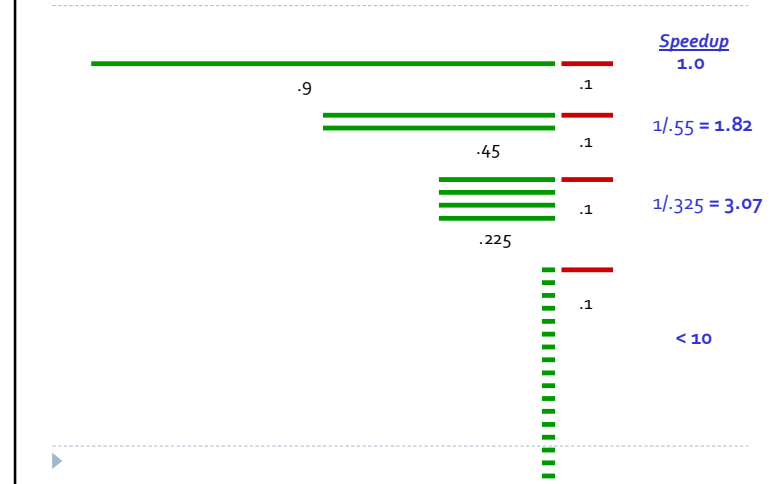
52

处理器性能瓶颈?



53

阿姆达尔定律和大规模并行



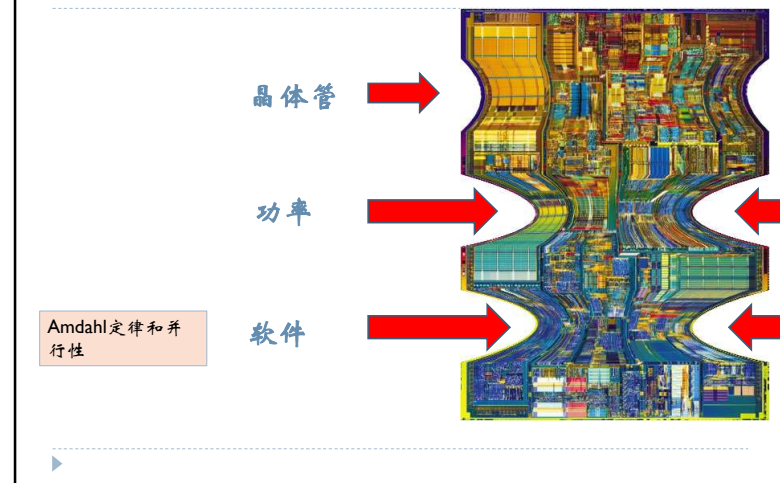
54

软件并行性的来源

- ▶ 多道程序
- ▶ 并行编程
- ▶ 并行化编译器
- ▶ 并行语言
- ▶ 串行代码动态并行机制 (例如, 通过动态编译)
- ▶ 投机多线程
- ▶ helper线程
- ▶

55

处理器性能瓶颈?



56

启示?

▶ 关注功率, 不是晶体管数

- ▶ 1. 通过优化“性能/瓦”来优化性能(不是“性能/晶体管数”)
- ▶ 2. Dark Silicon
 - ▶ 利用率墙—某些晶体管必须总是关闭
 - ▶ “怎样才能通过增加会被关掉的晶体管来增加处理器的价值?”
 - → heterogeneity



57

启示?

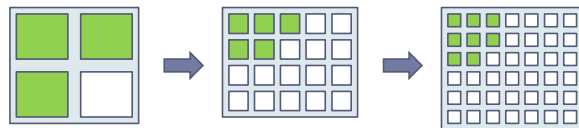
▶ 关注软件, 不是晶体管数

- ▶ 1. 许多核, 不是许多线程
- ▶ 2. 用任何可能的方式增加并行性—体系结构、编译器、编程语言、编程工具.....
- ▶ 3. 即使我们没有并行性, 仍然需要并行加速
- ▶ 4. 线程(运行这些线程的核或SMT上下文)是“免费的”



58

核 (和SMT上下文) 是“免费的”



- ▶ 如果问题是钉子, 锤子就是“多用些核”



59

软件, 不是晶体管/核数

▶ 启示

- ▶ 未来的机器获得性能的最佳方式: 体系结构使并行编程更简单(瓶颈不再是硬件, 而是程序员!)
- ▶ 快速的核间互联, 快速的一致性保证
- ▶ 硬件支持新的软件模式(transactional memory, 数据触发的线程.....)
- ▶ 硬件支持调试, 检测并行性.....



60

并行软件

- ▶ 由软件负责并行
 - ▶ 硬件和编译器可以跟上所需的速度
- ▶ 软件并行...
 - ▶ 在共享内存程序中
 - ▶ 启动单个进程并创建线程
 - ▶ 线程执行任务
 - ▶ 在分布式存储程序中
 - ▶ 启动多个进程
 - ▶ 进程执行任务

61

SPMD—单程序多数据

- ▶ SPMD程序由单个可执行文件组成，通过使用条件分支，该可执行文件可以像多个不同的程序一样运行

```
if (I'm thread process i)
    do this;
else
    do that;
```

62

写并程序序

- ▶ 在进程/线程之间分配工作
 - ▶ 每个进程/线程得到的工作量大致相同
 - ▶ 并且通信被最小化
- ▶ 安排进程/线程同步
- ▶ 安排进程/线程之间的通信

63

非确定性

```
...
printf ( "Thread %d > my_val = %d\n",
        my_rank , my_x );
...
```

Thread 1 > my_val = 19
Thread 0 > my_val = 7

Thread 0 > my_val = 7
Thread 1 > my_val = 19

64

内存一致性

```
my_val= Compute_val( my_rank );  
x += my_val;
```

Time	Core 0	Core 1
0	Finish assignment to my_val	In call to Compute_val
1	Load x = 0 into register	Finish assignment to my_val
2	Load my_val = 7 into register	Load x = 0 into register
3	Add my_val = 7 to x	Load my_val = 19 into register
4	Store x = 7	Add my_val to x
5	Start other work	Store x = 19

65

同步

- ▶ 竞争条件
- ▶ 临界区
- ▶ 互斥
- ▶ 互斥锁
- ▶ 忙等

```
my_val= Compute_val( my_rank );  
Lock(&add_my_val_lock);  
x += my_val;  
Unlock(&add_my_val_lock);
```

```
my_val= Compute_val( my_rank );  
if ( my_rank== 1)  
    while ( ! ok_for_1 ); /* Busy-wait loop */  
    x += my_val; /* Critical section */  
if ( my_rank== 0)  
    ok_for_1 = true; /* Let thread 1 update x */
```

66

输入输出

- ▶ 在分布式存储程序中，只有进程0会访问stdin
- ▶ 在共享内存程序中，只有主线程或线程0会访问stdin
- ▶ 在分布式存储和共享内存程序中，所有进程/线程都可以访问stdout和stderr
- ▶ 但是，由于输出到stdout的顺序不确定，在大多数情况下，除了调试输出之外，只有一个进程/线程用于所有输出到stdout的操作
- ▶ 调试输出应该总是包括生成输出的进程/线程的等级或id
- ▶ 只有单个进程/线程会尝试访问除stdin、stdout或stderr之外的任何单个文件
- ▶ 例如，每个进程/线程都可以打开自己的私有文件进行读写，但是没有两个进程/线程会打开同一个文件

67

并行编程的Foster方法论

- ▶ 1. 划分：把要执行的计算和计算操作的数据划分成小任务
 - ▶ 这里的重点应该是确定可以并行执行的任务
- ▶ 2. 通信：确定在上一步确定的任务中需要进行什么样的通信
- ▶ 3. 聚集或聚合：将第一步中确定的任务和通信组合成更大的任务
 - ▶ 例如，如果任务A必须在任务B执行之前执行，那么将它们聚合成一个复合任务可能是有意义的
- ▶ 4. 映射：将上一步中确定的复合任务分配给进程/线程
 - ▶ 这样做可以使通信最小化，并且每个进程/线程获得大致相同的工作量

68

性能

加速比

核数 = p

串行运行时间 = $T_{\text{串行}}$

并行运行时间 = $T_{\text{并行}}$

并行程序加速比

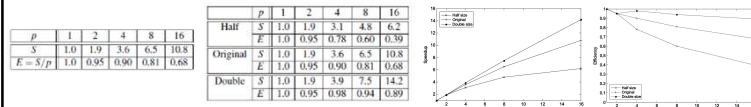
并行程序效率

$$T_{\text{并行}} = T_{\text{串行}} / p + T_{\text{overhead}}$$

$$T_{\text{并行}} = T_{\text{串行}} / p \quad \text{线性加速比}$$

$$S = T_{\text{串行}} / T_{\text{并行}}$$

$$E = S/p = [T_{\text{串行}} / T_{\text{并行}}] / p = T_{\text{串行}} / [p \cdot T_{\text{并行}}]$$



69

Amdahl's Law

除非几乎所有串行程序都被并行化，否则可能的加速将非常有限

不管有多少可用的核

例如：可以并行化90%的串行程序

无论使用多少个内核，并行化都是“完美”的

$T_{\text{串行}} = 20$ 秒

可并行化部分的运行时间为 $0.9 \times T_{\text{串行}} / p = 18/p$

“不可并行化”部分的运行时间为 $0.1 \times T_{\text{串行}} = 2$

总并行运行时间为 $T_{\text{并行}} = 0.9 \times T_{\text{串行}} / p + 0.1 \times T_{\text{串行}} = 18/p + 2$

$$S = \frac{T_{\text{serial}}}{0.9 \times T_{\text{serial}} / p + 0.1 \times T_{\text{serial}}} = \frac{20}{18/p + 2}$$

70

可扩展性

一般来说，如果一个问题规模增大后仍能够处理，那么它就是可扩展的

如果我们增加进程/线程的数量，并在不增加问题规模的情况下保持固定的效率，那么问题就具有强可扩展性

如果我们通过以与增加进程/线程数量相同的速度增加问题大小来保持固定的效率，那么问题是弱可扩展的

71

计算时间

时间是什么？

开始到结束？

计算哪些程序片段的时间？

CPU时间？

墙钟时间？

```
double start, finish;
...
start = Get_current_time();
/* Code that we want to time */
...
finish = Get_current_time();
printf("The elapsed time = %e seconds\n", finish-start);

private double start, finish;
...
start = Get_current_time();
/* Code that we want to time */
...
finish = Get_current_time();
printf("The elapsed time = %e seconds\n", finish-start);

shared double global_elapsed;
private double my_start, my_finish, my_elapsed;
/* Synchronize all processes/threads */
Barrier();
my_start = Get_current_time();
/* Code that we want to time */
...
my_finish = Get_current_time();
my_elapsed = my_finish - my_start;

/* Find the max across all processes/threads */
global_elapsed = Global_max(my_elapsed);
if (my_rank == 0)
    printf("The elapsed time = %e seconds\n", global_elapsed);
```

72

性能的不同层次

- ▶ 峰值性能 (Peak performance)
- ▶ LINPACK 性能
 - ▶ Avg. 80%
- ▶ Gordon Bell Prize 应用性能
 - ▶ ~30%
- ▶ 平均持续应用性能
 - ▶ <5%~10%
- ▶ HPCG
 - ▶ 1% ~ 5%

73

共享内存

- ▶ 动态线程
 - ▶ 主线程等待工作，创建新线程，当线程完成时，终止它们
 - ▶ 有效利用资源，但线程创建和终止非常耗时
- ▶ 静态线程
 - ▶ 线程池被创建并分配工作，但在清理之前不会终止
 - ▶ 更好的性能，但可能浪费系统资源

74

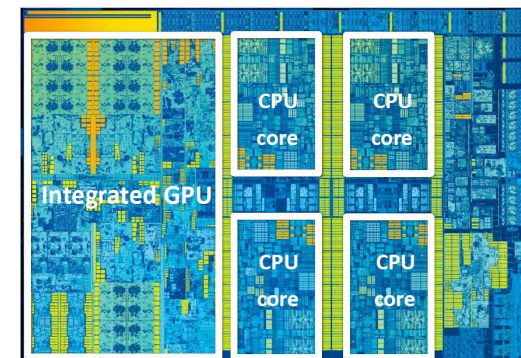
现代的并行系统



75

Intel Skylake (2015)(第六代 Core i7)

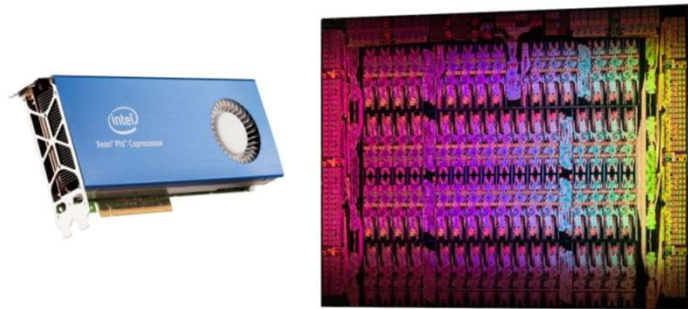
- ▶ 四核CPU + 多核GPU集成在一个芯片上



76

Intel Xeon Phi 7120A “协处理器”

- ▶ 61枚“简单”的x86内核(1.3 Ghz, 源自奔腾)
- ▶ 目标是作为超级计算应用的加速器



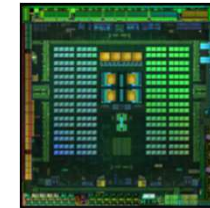
77

移动并行处理



▶ 苹果A12(在iPhone XR中)

- ▶ 4个CPU内核
- ▶ 4个GPU核心
- ▶ 神经网络引擎
- ▶ +更多



▶ 英伟达Tegra K1

- ▶ 四核ARM A57 CPU+ 4 ARM A53 CPU +
- ▶ NVIDIA GPU + 图像处理器...

78

程序员视角的性能

- ▶ 问题
如何让你的程序运行得更快?
- ▶ 2004年之前回答
等待6个月, 并购买一台新机器!
- ▶ 2004年以后的回答
需要写并行软件

79

例子

- ▶ 计算n个值, 并将它们相加
- ▶ 串行代码
- ▶ 有p个核, p比n小很多
- ▶ 每个核执行大约n/p个值的部分求和
- ▶ 在每个核完成代码执行后, 私有变量my_sum包含通过调用Compute_next_value计算出的值的总和
- ▶ 假如有8个核, n = 24, 那么对Compute_next_value的调用返回: 1,4,3, 9,2,8, 5,1,1, 5,2,7, 2,5,0, 4,1,8, 6,5,1, 2,3,9
- ▶ 一旦所有核都完成了对其私有my_sum的计算, 就通过将结果发送到指定的“主”核来形成全局和, 该“主”核添加最终结果

```
sum = 0;
for (i = 0; i < n; i++) {
    x = Compute_next_value(. . .);
    sum += x;
}

my_sum = 0;
my_first_i = . . . ;
my_last_i = . . . ;
for (my_i = my_first_i; my_i < my_last_i; my_i++) {
    my_x = Compute_next_value(. . .);
    my_sum += my_x;
}
```

```
if (!I'm the master core) {
    sum = my_x;
    for each core other than myself {
        receive value from core;
        sum += value;
    }
} else {
    send my_x to the master;
}
```

Core	0	1	2	3	4	5	6	7
my_sum	8	19	7	15	7	13	12	14

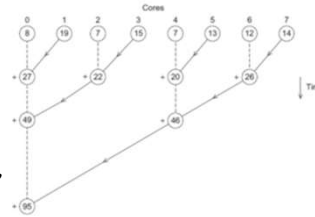
Global sum
8 + 19 + 7 + 15 + 7 + 13 + 12 + 14 = 95

Core	0	1	2	3	4	5	6	7
my_sum	95	19	7	15	7	13	12	14

80

改进并行算法

- ▶ 不要让主核做所有的工作，在其他核之间并行执行
- ▶ 将奇数和偶数编号的核配对，核0将其结果与核1的结果相加，核2将其结果与核3的结果相加，等等
- ▶ 对2的倍数编号的核重复该过程，核0将核2的结果相加，核4将核6的结果相加，依此类推
- ▶ 可被4整除的核重复该过程，以此类推，直到核0具有最终结果



分析

- ▶ 在第一个例子中，主核执行7次接收和7次加法
- ▶ 在第二个例子中，主核执行3次接收和3次加法
- ▶ 加速超过了2倍
- ▶ 核数量越多，差异就越明显
- ▶ 如果我们有1000个核
- ▶ 第一个例子要求主设备执行999次接收和999次加法
- ▶ 第二个例子只需要10次接收和10次加法
- ▶ 这几乎是100倍的加速

81

82

如何写并行程序

任务并行

- ▶ 将为解决问题而执行的各种任务划分到核中

数据并行

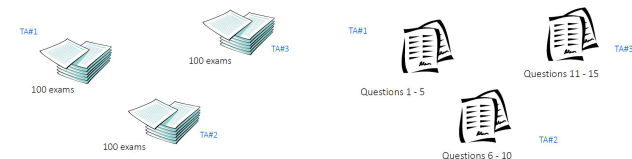
- ▶ 在核之间划分用于解决问题的数据
- ▶ 每个核对其数据部分执行类似的操作

```
sum = 0;
for (i = 0; i < n; i++) {
    x = Compute_next_value(. . .);
    sum += x;
}
```

```
if (I'm the master core) {
    sum = my_x;
    for each core other than myself {
        receive value from core;
        sum += value;
    }
} else {
    send my_x to the master;
}
```



P教授
15 个问题
300 份试卷



83

84

协同

- 核通常需要协同工作
- 通信——一个或多个核将它们当前的部分和发送到另一个核
- 负载均衡——在核之间平均分配工作，这样一个核就不会负载过重
- 同步——因为每个核都以自己的速度工作，所以要确保核不会领先其他核太多

85

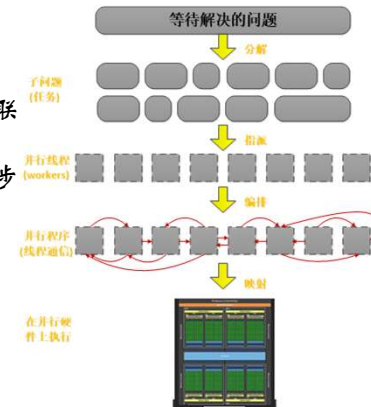
创建并行程序

思考：

- 1、明确可以并行的任务
- 2、划分任务（包括与任务关联的数据）
- 3、处理数据读写、通信和同步

并行的主要目标是加速比（speedup）

$$\text{speedup}_p = T_1 / T_p$$



这些工作由程序员、系统(编译器、运行时或者硬件)分别或者共同负责完成

86

分解

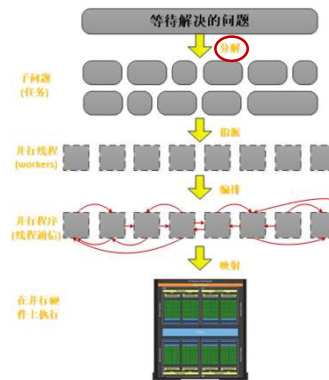
- 将问题分解成可以并行执行的任务
 - 分解是动态的
 - 随着程序的执行，需要识别新的任务

- 主要思路：想办法创建足够多的任务使得系统中所有的执行单元保持忙碌

分解的关键：识别依赖性

阿姆达尔定律 (Amdahl's Law) —— 依赖性限制了最大的并行加速比

设S为串行程序中由于依赖性的限制而无法并行的部分，则并行执行可以获得的最大加速比 $\leq 1/S$



这些工作由程序员、系统(编译器、运行时或者硬件)分别或者共同负责完成

87

一个简单的例子

- 设想一个分两步计算处理的 $N \times N$ 图像
 - 第一步：将所有像素点的亮度加倍（每个像素的计算互相独立）
 - 第二步：计算所有像素值的平均值

串行执行

每一步执行时间为 N^2 ，总执行时间为 $2N^2$

初步尝试并行

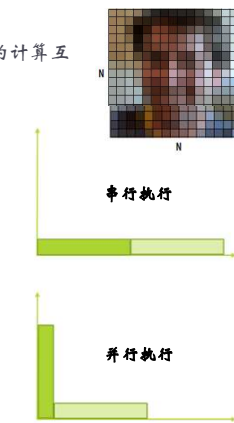
策略

- 第一步：并行执行，执行时间为 N^2/p
- 第二步：串行执行，执行时间为 N^2
- 总执行时间为 $N^2/p + N^2$

总体性能

$$\text{Speedup} \leq 2N^2 / (N^2/p + N^2)$$

$$\text{Speedup} \leq 2$$



88

进一步并行

策略

- ▶ 第一步：并行执行，执行时间为 N^2/p
- ▶ 第二步：并行计算部分和，串行计算总和，执行时间为 $N^2/p+p$

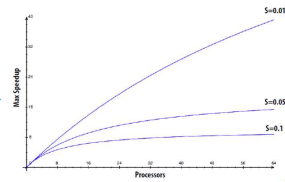
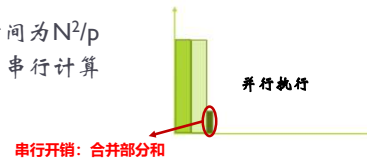
总体性能

$$\text{Speedup} \leq 2N^2/(2N^2/p+p)$$

当 $N \gg p$ 时, $\text{Speedup} \rightarrow p$

阿姆达尔定律 (Amdahl's Law)

设 S 为串行程序中由于依赖性的限制而无法并行的部分
则在 p 个处理器上并行执行可以获得的最大加速比
 $\text{Speedup} \leq 1/(S+(1-S)/p)$



89

分解

谁来负责任务分解?

- ▶ 通常来说, 由程序员负责

串行程序的自动任务分解始终是一个具有挑战性的研究问题 (通常相当难)

- ▶ 编译器必须分析程序, 识别依赖性
 - ▶ 有些数据依赖在编译时无法知晓
- ▶ 研究人员在简单的循环嵌套上取得了一定的成功
- ▶ 能够处理复杂、通用代码的“魔法并行编译器”还有很多工作要做

90